ΟΛΟΚΛΗΡΩΜΕΝΟ ΣΥΣΤΗΜΑ ΔΙΑΧΕΙΡΗΣΗΣ ΚΙΝΗΜΑΤΟΓΡΑΦΟΥ

ΤΕΧΝΟΛΟΓΙΑ ΛΟΓΙΣΜΙΚΟΥ



ΧΡΗΣΤΟΣ ΤΡΙΑΝΤΑΦΥΛΛΑΚΗΣ

ΝΙΚΟΛΑΟΣ ΧΡΙΣΤΟΠΟΥΛΟΣ

Πανεπιστήμιο Αιγαίου

Τμήμα Μηχανικών Πληροφοριακών και Επικοινωνιακών Συστημάτων

ΤΕΧΝΟΛΟΓΙΑ ΛΟΓΙΣΜΙΚΟΥ

Διδάσκων: Κυριάκος Κρητικός

Εργαστηριακός διδάσκων: Φώτιος Κατσίφας

Χρήστος Τριανταφυλλάκης: icsd22219

Νικόλαος Χριστόπουλος: icsd22250

Σάμος 14/01/2026

Περίληψη

Η παρούσα εργασία αφορά την ανάλυση, τον σχεδιασμό και την υλοποίηση ενός πληροφοριακού συστήματος διαχείρισης κινηματογραφικών προγραμμάτων και προβολών. Το σύστημα υποστηρίζει τη δημιουργία και διαχείριση κινηματογραφικών σεζόν (programs), την υποβολή και αξιολόγηση προβολών (screenings), καθώς και τη σταδιακή μετάβαση των προγραμμάτων από τη φάση της δημιουργίας έως και την τελική ανακοίνωσή τους στο κοινό. Η λειτουργικότητα του συστήματος βασίζεται σε ρόλους χρηστών (Visitor, User, Programmer, Staff, Submitter), με αυστηρό έλεγχο πρόσβασης και δικαιωμάτων, ώστε να διασφαλίζεται η ορθή ροή και ακεραιότητα της διαδικασίας.

Η υλοποίηση του συστήματος πραγματοποιήθηκε με τη χρήση της γλώσσας προγραμματισμού **Python**, ενώ η ανάπτυξη των RESTful web services έγινε με το πλαίσιο **FastAPI**. Για την αποθήκευση και διαχείριση των δεδομένων χρησιμοποιήθηκε σχεσιακή βάση δεδομένων, η οποία υλοποιήθηκε μέσω του **XAMPP**. Το σύστημα δίνει έμφαση στην αξιοπιστία, την ασφάλεια, την ταχύτητα εκτέλεσης και τη σωστή διαχείριση σφαλμάτων, ακολουθώντας βασικές αρχές της μηχανικής λογισμικού και της ανάπτυξης σύγχρονων υπηρεσιοστρεφών εφαρμογών.

Περιεχόμενα

1. Οργάνωση

1.1 Βασικοί ρόλοι στην σχεδίαση και ανάπτυξη και ποια μέλη της ομάδας ανέλαβαν αυτούς τους ρόλους

1.2 Βασικές δραστηριότητες/στάδια ανάπτυξης και διάρκειες αυτών

2.Απαιτήσεις

2.1.Λειτουργικές απαιτήσεις

2.2.Μη λειτουργικές απαιτήσεις

3.Διαγράμματα-σχεδίαση

3.1 Διάγραμμα Συστατικών (Component Diagram)

3.2 Διάγραμμα Περιβάλλοντος (Context Diagram)

3.3 Διαγράμματα Περιπτώσεων Χρήσης (Use Case Diagrams)

3.4 Διαγράμματα Δραστηριοτήτων (Activity Diagrams)

3.5 Διαγράμματα Ακολουθίας (Sequence Diagrams)

3.6 Διάγραμμα Οντοτήτων-Συσχετίσεων (Entity-Relationship (ER) diagram)

4. Υλοποίηση Συστήματος / Εφαρμογής

4.1 Περιβάλλον Ανάπτυξης και Έλεγχος REST API

4.2 Υλοποίηση και Σχολιασμός

5.Συμπεράσματα

5.1.Εμπειρία που αποκτήθηκε από το έργο

5.2. Προβλήματα και δυσκολίες που αντιμετωπίστηκαν

5.3. Βέλτιστες πρακτικές και συμπεράσματα για μελλοντικά έργα

6.Πηγές

7.GitHub link

1.Οργάνωση

1.1 Βασικοί ρόλοι στη σχεδίαση και ανάπτυξη και κατανομή στην ομάδα

Η ανάπτυξη του συστήματος πραγματοποιήθηκε από ομάδα δύο ατόμων, με συνεργατική προσέγγιση σε όλα τα στάδια της διαδικασίας ανάπτυξης λογισμικού. Και τα δύο μέλη της ομάδας συμμετείχαν από κοινού στην ανάλυση απαιτήσεων και στον σχεδιασμό του συστήματος, μέσω της μελέτης της εκφώνησης και της δημιουργίας των διαγραμμάτων UML.

Κατά τη φάση της υλοποίησης, υπήρξε διακριτός αλλά συμπληρωματικός καταμερισμός εργασιών. Το πρώτο μέλος της ομάδας έδωσε μεγαλύτερη έμφαση στην παραγωγή και υλοποίηση του πηγαίου κώδικα, αναλαμβάνοντας την ανάπτυξη των βασικών λειτουργιών και των RESTful υπηρεσιών του συστήματος. Το δεύτερο μέλος επικεντρώθηκε κυρίως στον έλεγχο της εφαρμογής, στη δοκιμή των λειτουργιών και στην επαλήθευση της ορθής συμπεριφοράς των endpoints, εντοπίζοντας και αναφέροντας σφάλματα ή ασυνέπειες. Η τεκμηρίωση του έργου και η τελική σύνθεση της αναφοράς πραγματοποιήθηκαν σε συνεργασία και από τα δύο μέλη, διασφαλίζοντας τη συνοχή και την πληρότητα της εργασίας.

1.2 Βασικές δραστηριότητες / στάδια ανάπτυξης και διάρκειες

Η ανάπτυξη του συστήματος ακολούθησε διακριτά στάδια, σύμφωνα με τις βασικές αρχές της μηχανικής λογισμικού. Τα κύρια στάδια και η εκτιμώμενη διάρκειά τους παρουσιάζονται παρακάτω:

- **Ανάλυση απαιτήσεων**
  Μελέτη της εκφώνησης, κατανόηση των λειτουργικών και μη λειτουργικών απαιτήσεων και καθορισμός των ρόλων χρηστών.
  Διάρκεια: ~6 ώρες

- **Σχεδίαση συστήματος**
  Δημιουργία των διαγραμμάτων UML (Context, Use Case, Activity, Sequence, ER) και καθορισμός της αρχιτεκτονικής του συστήματος.
  Διάρκεια: ~15 ώρες

- **Υλοποίηση (παραγωγή κώδικα)**
  Ανάπτυξη του backend συστήματος με χρήση Python και FastAPI, καθώς και υλοποίηση της σύνδεσης με τη βάση δεδομένων.
  Διάρκεια: ~17 ώρες

- **Έλεγχος και δοκιμές (testing)**
  Έλεγχος της λειτουργικότητας των REST endpoints μέσω Swagger UI, εντοπισμός και διόρθωση σφαλμάτων.
  Διάρκεια: ~10 ώρες

- **Τεκμηρίωση και τελικές βελτιώσεις**
  Συγγραφή της τελικής αναφοράς και μικρές διορθώσεις στον κώδικα και στη δομή του έργου.
  Διάρκεια: ~8 ώρες

Συνολικός εκτιμώμενος χρόνος ανάπτυξης: ~56 ώρες

2.Απαιτήσεις

2.1. Λειτουργικές Απαιτήσεις

1. Το σύστημα πρέπει να επιτρέπει την αυθεντικοποίηση χρηστών μέσω μηχανισμού σύνδεσης (login).
Η απαίτηση αυτή ορίζει ότι το σύστημα οφείλει να υποστηρίζει μηχανισμό εισόδου χρηστών με χρήση διαπιστευτηρίων (π.χ. όνομα χρήστη και κωδικό). Η αυθεντικοποίηση είναι απαραίτητη ώστε το σύστημα να αναγνωρίζει την ταυτότητα του χρήστη και να μπορεί να εφαρμόζει περιορισμούς πρόσβασης στις λειτουργίες του, σύμφωνα με τον ρόλο του.

2. Το σύστημα πρέπει να υποστηρίζει τη διαχείριση χρηστών με διαφορετικούς ρόλους (Programmer, Staff, Submitter, Visitor).
Το σύστημα πρέπει να υποστηρίζει πολλαπλούς ρόλους χρηστών, όπου κάθε ρόλος καθορίζει διαφορετικά δικαιώματα και ευθύνες. Οι ρόλοι αυτοί επιτρέπουν την ορθή κατανομή αρμοδιοτήτων (π.χ. έγκριση προβολών, αξιολόγηση, υποβολή) και αποτρέπουν μη εξουσιοδοτημένες ενέργειες.

3. Το σύστημα πρέπει να επιτρέπει τη δημιουργία, επεξεργασία και διαγραφή προγραμμάτων κινηματογραφικών εκδηλώσεων.
Η απαίτηση αυτή αφορά τη διαχείριση των κινηματογραφικών προγραμμάτων (seasons). Το σύστημα πρέπει να παρέχει λειτουργίες CRUD (Create, Read, Update, Delete) για τα προγράμματα, με κατάλληλους περιορισμούς (π.χ. διαγραφή μόνο σε συγκεκριμένη κατάσταση).

4. Το σύστημα πρέπει να επιτρέπει την ανάθεση χρηστών σε προγράμματα με συγκεκριμένο ρόλο.
Το σύστημα πρέπει να δίνει τη δυνατότητα συσχέτισης χρηστών με προγράμματα και ανάθεσης συγκεκριμένου ρόλου εντός αυτών. Έτσι, ένας χρήστης μπορεί να έχει διαφορετικό ρόλο σε διαφορετικά προγράμματα, διατηρώντας τη λειτουργική ευελιξία του συστήματος.

5. Το σύστημα πρέπει να υποστηρίζει τη δημιουργία και επεξεργασία προβολών (screenings) εντός ενός προγράμματος.
Η απαίτηση αυτή καθορίζει ότι κάθε προβολή πρέπει να ανήκει σε κάποιο πρόγραμμα και να μπορεί να δημιουργηθεί ή να τροποποιηθεί κατά τα αρχικά στάδια του κύκλου ζωής της, πριν την υποβολή ή έγκρισή της.

6. Το σύστημα πρέπει να επιτρέπει την υποβολή προβολών από τους χρήστες που έχουν το δικαίωμα υποβολής.
Το σύστημα πρέπει να επιτρέπει μόνο σε χρήστες με ρόλο Submitter να υποβάλλουν προβολές για αξιολόγηση. Η λειτουργία αυτή σηματοδοτεί τη μετάβαση της προβολής από κατάσταση επεξεργασίας σε επίσημη υποβολή.

7. Το σύστημα πρέπει να επιτρέπει την απόσυρση προβολών πριν την τελική έγκριση.
Η απαίτηση αυτή διασφαλίζει ότι ένας submitter μπορεί να αποσύρει μια προβολή εφόσον αυτή δεν έχει ακόμη εγκριθεί, αποτρέποντας τη συνέχιση της διαδικασίας αξιολόγησης για μη επιθυμητές υποβολές.

8. Το σύστημα πρέπει να υποστηρίζει την ανάθεση προσωπικού (staff) σε προβολές.
Το σύστημα πρέπει να επιτρέπει την ανάθεση συγκεκριμένου μέλους staff σε κάθε προβολή, το οποίο θα είναι υπεύθυνο για την αξιολόγηση και διαχείρισή της. Η ανάθεση αυτή γίνεται σε συγκεκριμένο στάδιο του κύκλου ζωής του προγράμματος.

9. Το σύστημα πρέπει να επιτρέπει την αξιολόγηση προβολών από χρήστες με ρόλο staff, μέσω σχολίων και βαθμολογίας.
Η απαίτηση αυτή ορίζει ότι τα μέλη staff μπορούν να παρέχουν ποιοτική (σχόλια) και ποσοτική (βαθμολογία) αξιολόγηση για τις προβολές που τους έχουν ανατεθεί, υποστηρίζοντας τη διαδικασία λήψης αποφάσεων.

10. Το σύστημα πρέπει να επιτρέπει την έγκριση ή απόρριψη προβολών από τον χρήστη με ρόλο programmer.

Το σύστημα πρέπει να παρέχει στους programmers τη δυνατότητα να λαμβάνουν τελικές αποφάσεις σχετικά με την αποδοχή ή απόρριψη προβολών, βάσει των αξιολογήσεων και των κριτηρίων του προγράμματος.

11. Το σύστημα πρέπει να υποστηρίζει τελική υποβολή προβολών και την ένταξή τους στο τελικό πρόγραμμα.
Μετά την έγκριση, το σύστημα πρέπει να επιτρέπει την τελική υποβολή της προβολής και την ένταξή της στο οριστικό πρόγραμμα προβολών, όπου περιορίζονται περαιτέρω τροποποιήσεις.

12. Το σύστημα πρέπει να υποστηρίζει διαφορετικές καταστάσεις (status) για προγράμματα και προβολές, σύμφωνα με τον κύκλο ζωής τους.
Το σύστημα πρέπει να υλοποιεί μηχανισμό καταστάσεων (state machine) για προγράμματα και προβολές, ώστε να ελέγχεται ποια ενέργεια επιτρέπεται σε κάθε στάδιο και να διασφαλίζεται η ορθότητα της ροής εργασιών.

13. Το σύστημα πρέπει να επιτρέπει την αναζήτηση προγραμμάτων με βάση πολλαπλά κριτήρια.
Το σύστημα πρέπει να παρέχει λειτουργία αναζήτησης που να υποστηρίζει πολλαπλά φίλτρα, όπως όνομα, περιγραφή, ημερομηνίες, τίτλο ταινίας και αίθουσα, διευκολύνοντας την εύρεση συγκεκριμένων προγραμμάτων.

14. Η αναζήτηση προγραμμάτων πρέπει να εφαρμόζει σημασιολογία AND.
Όταν ο χρήστης παρέχει περισσότερα από ένα κριτήρια αναζήτησης, το σύστημα πρέπει να επιστρέφει μόνο τα αποτελέσματα που ικανοποιούν **όλα** τα κριτήρια ταυτόχρονα, εξασφαλίζοντας ακριβή αποτελέσματα.

15. Τα αποτελέσματα της αναζήτησης πρέπει να φιλτράρονται με βάση τον ρόλο του χρήστη και να ταξινομούνται.
Το σύστημα πρέπει να εμφανίζει μόνο τα προγράμματα που επιτρέπεται να δει ο εκάστοτε χρήστης, σύμφωνα με τον ρόλο του, και να ταξινομεί τα αποτελέσματα πρώτα χρονικά και στη συνέχεια αλφαβητικά.

2.2. Μη Λειτουργικές Απαιτήσεις

1.Το σύστημα πρέπει να υλοποιείται ως RESTful API.

Η αρχιτεκτονική του συστήματος πρέπει να βασίζεται σε RESTful υπηρεσίες, επιτρέποντας ανεξαρτησία πελάτη–διακομιστή, επεκτασιμότητα και ευκολία ενσωμάτωσης με άλλα συστήματα.

2. Το σύστημα πρέπει να παρέχει τεκμηρίωση μέσω OpenAPI/Swagger.

Η απαίτηση αυτή διασφαλίζει ότι όλες οι διαθέσιμες υπηρεσίες του API είναι τεκμηριωμένες, διευκολύνοντας τη χρήση, τη δοκιμή και τη συντήρηση του συστήματος.

3. Το σύστημα πρέπει να υποστηρίζει έλεγχο πρόσβασης βάσει ρόλων χρηστών.

Το σύστημα πρέπει να εφαρμόζει μηχανισμούς εξουσιοδότησης (authorization), ώστε κάθε λειτουργία να είναι προσβάσιμη μόνο από τους κατάλληλους ρόλους.

4. Το σύστημα πρέπει να επιστρέφει κατάλληλους κωδικούς HTTP.

Κάθε αίτημα προς το API πρέπει να επιστρέφει τον σωστό HTTP status code (π.χ. 200, 400, 401, 403, 404), επιτρέποντας στους πελάτες να κατανοούν την έκβαση της κλήσης.

5. Το σύστημα πρέπει να διασφαλίζει την ακεραιότητα και τη συνέπεια των δεδομένων.

Το σύστημα πρέπει να χρησιμοποιεί μηχανισμούς ελέγχου εγκυρότητας και συναλλαγών (transactions), ώστε τα δεδομένα στη βάση να παραμένουν ορθά και συνεπή.

6. Το σύστημα πρέπει να χειρίζεται σωστά σφάλματα και εξαιρέσεις.

Το σύστημα οφείλει να εντοπίζει και να διαχειρίζεται σφάλματα, παρέχοντας κατανοητά μηνύματα λάθους χωρίς να αποκαλύπτει ευαίσθητες πληροφορίες.

7. Το σύστημα πρέπει να έχει αποδεκτό χρόνο απόκρισης.

Οι υπηρεσίες του συστήματος πρέπει να ανταποκρίνονται σε εύλογο χρονικό διάστημα, εξασφαλίζοντας ικανοποιητική εμπειρία χρήστη ακόμη και υπό αυξημένο φόρτο.

8. Το σύστημα πρέπει να είναι επεκτάσιμο.

Η αρχιτεκτονική του συστήματος πρέπει να επιτρέπει την προσθήκη νέων λειτουργιών ή ρόλων στο μέλλον χωρίς σημαντικές αλλαγές στον υπάρχοντα κώδικα.

9. Το σύστημα πρέπει να καταγράφει κρίσιμες ενέργειες χρηστών.

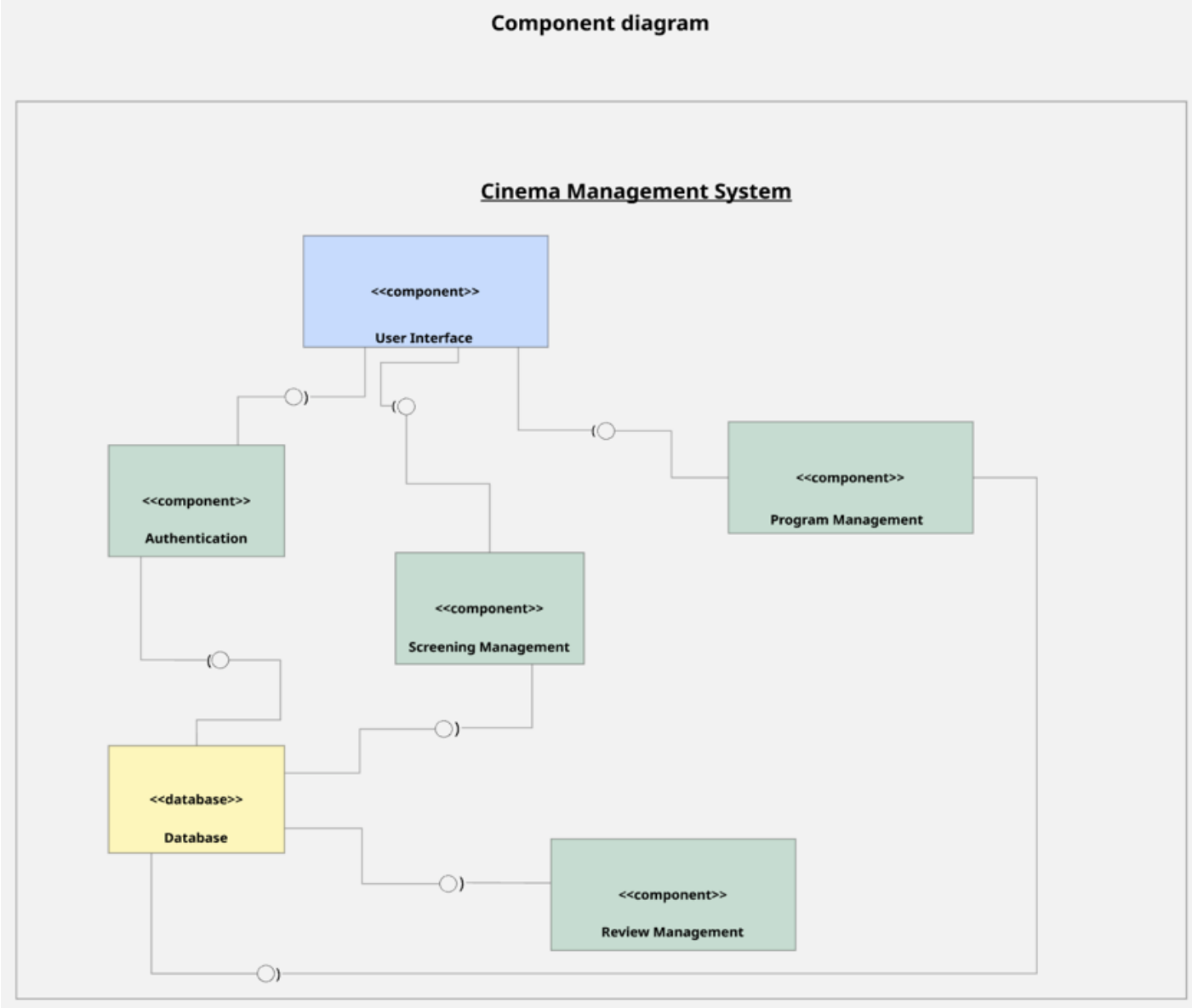
Το σύστημα πρέπει να διατηρεί αρχεία καταγραφής (logs) για σημαντικές ενέργειες, όπως εγκρίσεις ή απορρίψεις, ώστε να υποστηρίζεται ο έλεγχος και η ιχνηλασιμότητα.

10. Το σύστημα πρέπει να αποθηκεύει τους κωδικούς χρηστών με ασφαλή τρόπο.

Οι κωδικοί χρηστών δεν πρέπει να αποθηκεύονται σε απλό κείμενο αλλά με χρήση ασφαλών τεχνικών κατακερματισμού (hashing), ενισχύοντας την ασφάλεια του συστήματος.

3.Διαγράμματα-σχεδίαση

3.1 Διάγραμμα Συστατικών (Component Diagram)



Σχολιασμός Διαγράμματος Συστατικών (Component Diagram)

Το διάγραμμα συστατικών παρουσιάζει τη βασική αρχιτεκτονική του συστήματος διαχείρισης κινηματογράφου (Cinema Management System) και τον τρόπο με τον οποίο αλληλεπιδρούν τα κύρια δομικά του στοιχεία. Η σχεδίαση ακολουθεί μια διαχωρισμένη αρχιτεκτονική, όπου κάθε συστατικό αναλαμβάνει συγκεκριμένες ευθύνες, διευκολύνοντας τη συντήρηση και την επεκτασιμότητα του συστήματος. Το **User Interface** αποτελεί το σημείο αλληλεπίδρασης του χρήστη με το σύστημα και είναι υπεύθυνο για την αποστολή αιτημάτων προς τις αντίστοιχες υπηρεσίες του backend. Μέσω αυτού, οι χρήστες μπορούν να εκτελούν ενέργειες όπως η προβολή, δημιουργία και διαχείριση προγραμμάτων και προβολών.

Το **Authentication** είναι υπεύθυνο για τη διαχείριση της αυθεντικοποίησης και εξουσιοδότησης των χρηστών. Ελέγχει τα διαπιστευτήρια εισόδου και καθορίζει τα δικαιώματα πρόσβασης βάσει ρόλων, διασφαλίζοντας την ασφάλεια του συστήματος.

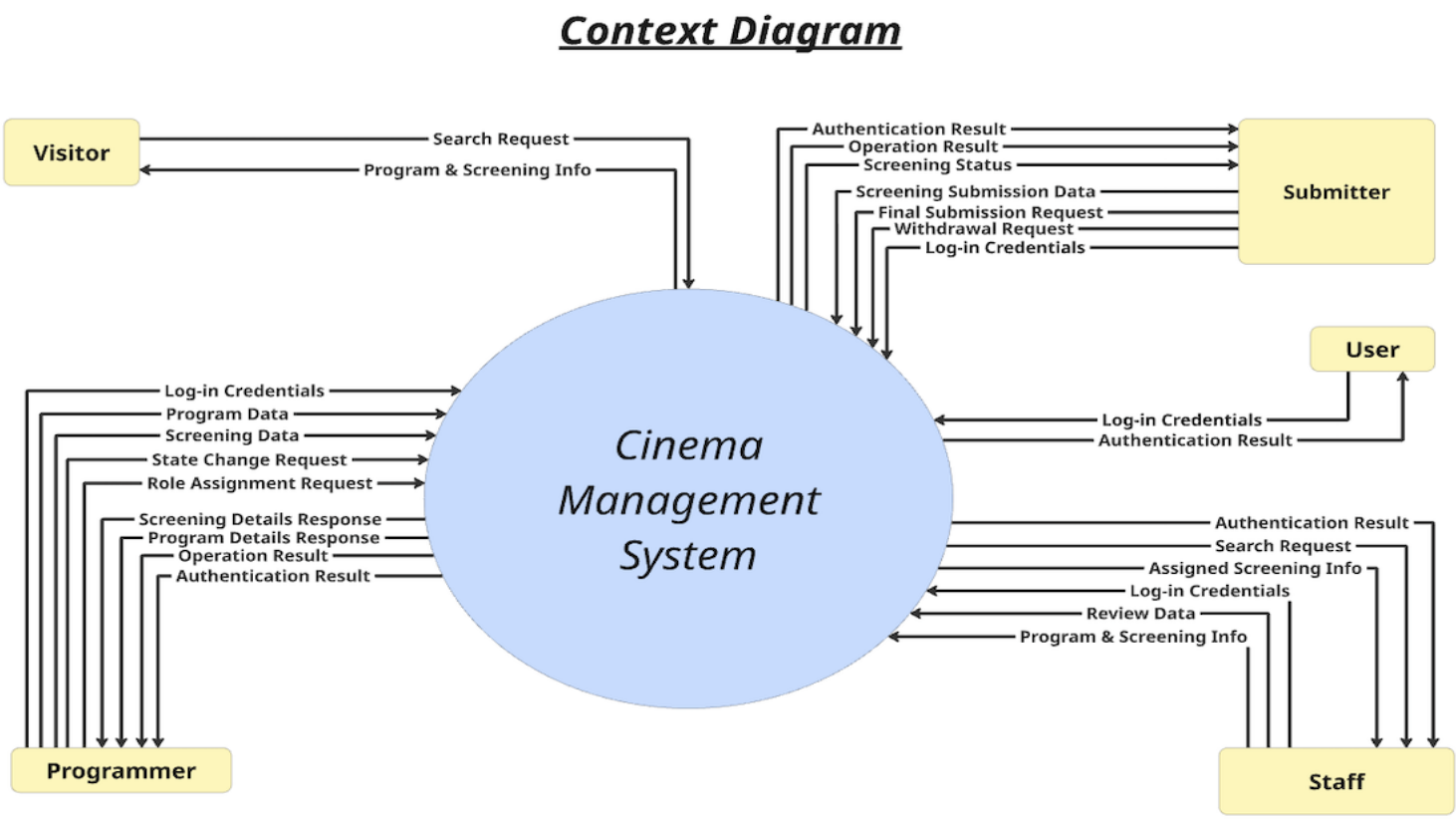
Το **Program Management** χειρίζεται τη διαχείριση των κινηματογραφικών προγραμμάτων (seasons), συμπεριλαμβανομένης της δημιουργίας, τροποποίησης και της μετάβασης των προγραμμάτων μεταξύ των διαφόρων καταστάσεων του κύκλου ζωής τους.

Το **Screening Management** είναι υπεύθυνο για τη διαχείριση των προβολών, όπως η δημιουργία, υποβολή, απόσυρση και τελική αποδοχή τους, σύμφωνα με την κατάσταση του εκάστοτε προγράμματος.

Το **Review Management** υποστηρίζει τη διαδικασία αξιολόγησης των προβολών από το προσωπικό (STAFF), επιτρέποντας την καταχώρηση σχολίων και βαθμολογιών.

Τέλος, η **Database** αποτελεί το επίπεδο αποθήκευσης δεδομένων του συστήματος και χρησιμοποιείται από όλα τα βασικά συστατικά για την αποθήκευση και ανάκτηση πληροφοριών. Η επικοινωνία μεταξύ των συστατικών και της βάσης δεδομένων διασφαλίζει τη συνέπεια και την ακεραιότητα των δεδομένων.Συνολικά, το διάγραμμα αποτυπώνει μια καθαρή και αρθρωτή αρχιτεκτονική, η οποία υποστηρίζει την υλοποίηση του συστήματος με RESTful υπηρεσίες και ευθυγραμμίζεται με τις αρχές της μηχανικής λογισμικού.

3.2 Διάγραμμα Περιβάλλοντος (Context Diagram)



Σχολιασμός Διαγράμματος Περιβάλλοντος (Context Diagram)

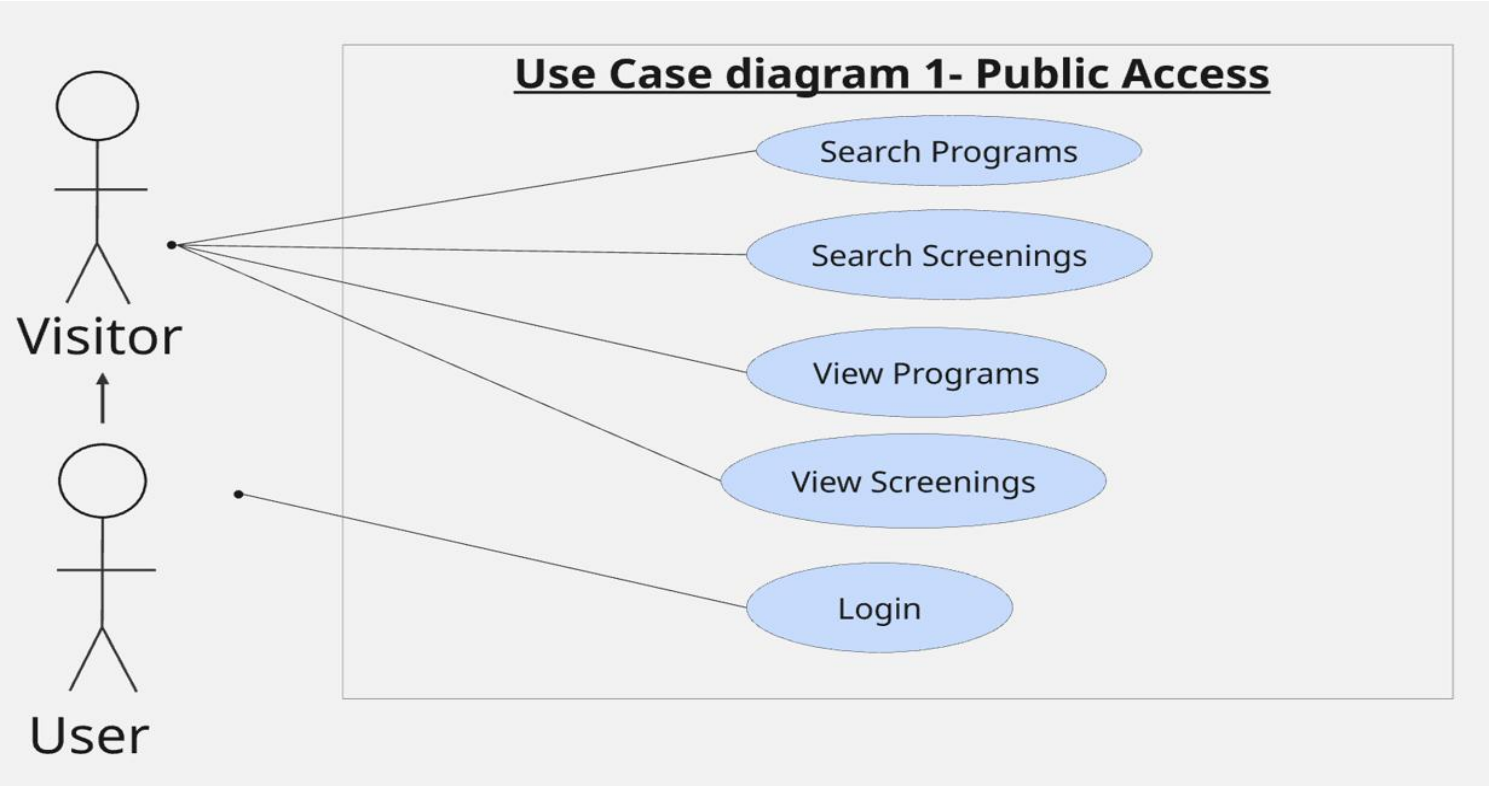
Το διάγραμμα περιβάλλοντος παρουσιάζει το Cinema Management System ως ένα ενιαίο σύστημα και απεικονίζει τις αλληλεπιδράσεις του με τους εξωτερικούς χρήστες και ρόλους που το περιβάλλουν. Στο διάγραμμα εμφανίζονται όλοι οι βασικοί ρόλοι του συστήματος (Visitor, User, Submitter, Staff και Programmer), καθώς και οι κύριες ροές δεδομένων μεταξύ αυτών και του συστήματος.

Ο ρόλος του Visitor περιορίζεται σε αιτήματα αναζήτησης και προβολής δημόσιων πληροφοριών προγραμμάτων και προβολών, χωρίς δυνατότητα αυθεντικοποίησης ή διαχείρισης. Ο User μπορεί να αυθεντικοποιηθεί και να αποκτήσει βασικές λειτουργίες, όπως

είσοδο στο σύστημα και προβολή πληροφοριών. Ο Submitter αλληλεπιδρά με το σύστημα για την υποβολή, απόσυρση και τελική υποβολή προβολών, λαμβάνοντας αντίστοιχες απαντήσεις σχετικά με την κατάσταση των αιτημάτων του. Ο Staff επικοινωνεί με το σύστημα για την ανάκτηση ανατεθειμένων προβολών και την υποβολή αξιολογήσεων, ενώ ο Programmer διαθέτει το πληρέστερο σύνολο αλληλεπιδράσεων, καθώς διαχειρίζεται προγράμματα, ρόλους, καταστάσεις και προβολές. Το διάγραμμα αποτυπώνει με σαφήνεια τα όρια του συστήματος και τις ροές πληροφορίας, αναδεικνύοντας τον ρόλο της αυθεντικοποίησης και του ελέγχου πρόσβασης ως βασικά στοιχεία της συνολικής λειτουργίας του συστήματος.

3.3 Διαγράμματα Περιπτώσεων Χρήσης (Use Case Diagrams)

1ο διάγραμμα Public Access

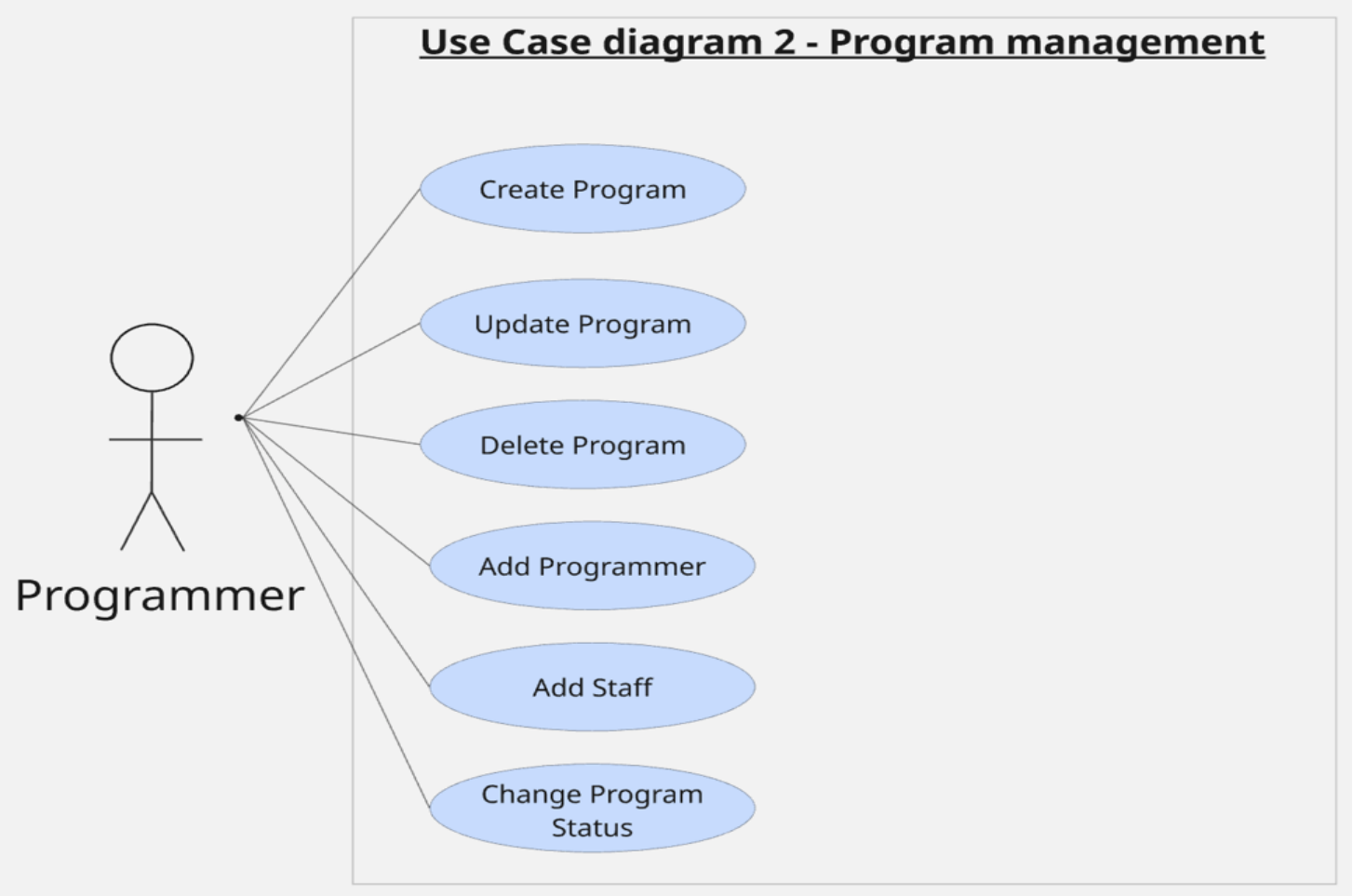


Σχολιασμός Διαγράμματος Περιπτώσεων Χρήσης – Δημόσια Πρόσβαση (Use Case Diagram 1)

Το διάγραμμα περιπτώσεων χρήσης για τη δημόσια πρόσβαση απεικονίζει τις βασικές λειτουργίες του συστήματος που είναι διαθέσιμες σε μη αυθεντικοποιημένους και βασικούς χρήστες. Κύριος δρών στο διάγραμμα είναι ο Visitor, ο οποίος αντιπροσωπεύει οποιονδήποτε ανώνυμο χρήστη που επισκέπτεται το σύστημα χωρίς να έχει συνδεθεί. Ο Visitor έχει τη δυνατότητα να αναζητά και να προβάλλει προγράμματα και προβολές μέσω των περιπτώσεων χρήσης *Search Programs*, *Search Screenings*, *View Programs* και *View Screenings*, λαμβάνοντας αποκλειστικά δημόσια διαθέσιμες πληροφορίες.

Επιπλέον, στο διάγραμμα εμφανίζεται και ο User, ο οποίος αποτελεί εξειδίκευση του Visitor και διαθέτει τη δυνατότητα αυθεντικοποίησης μέσω της περίπτωσης χρήσης *Login*. Μετά την επιτυχή σύνδεση, ο User μπορεί να αποκτήσει επιπλέον δικαιώματα και ρόλους, ανάλογα με τη συσχέτισή του με συγκεκριμένα προγράμματα. Το διάγραμμα αποτυπώνει με σαφήνεια τον διαχωρισμό μεταξύ δημόσιας και ελεγχόμενης πρόσβασης, αναδεικνύοντας τη σημασία της αυθεντικοποίησης ως προϋπόθεση για την εκτέλεση πιο προχωρημένων λειτουργιών του συστήματος.

2ο διάγραμμα Program Management



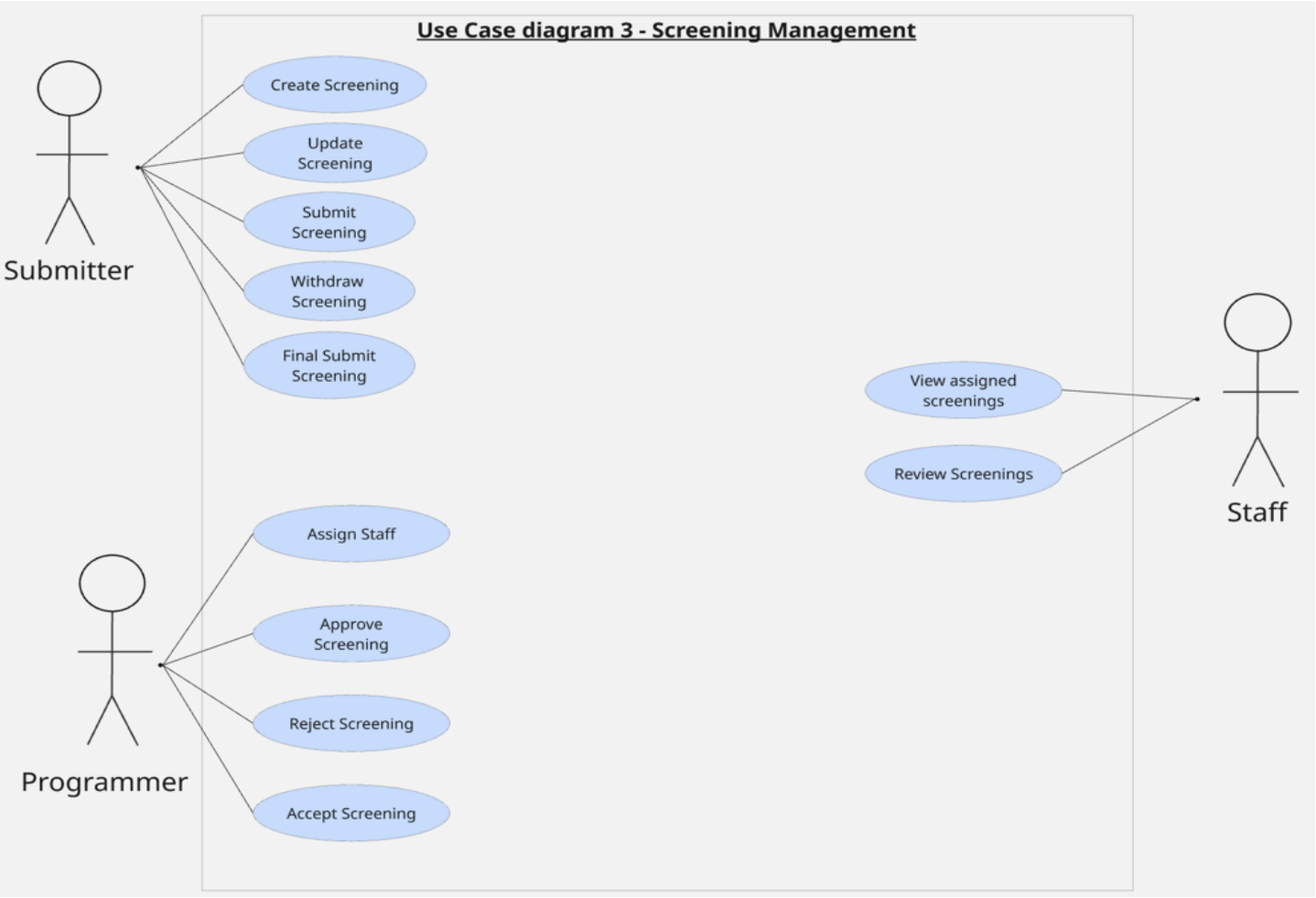
Σχολιασμός Διαγράμματος Περιπτώσεων Χρήσης – Διαχείριση Προγραμμάτων (Use Case Diagram 2)

Το διάγραμμα περιπτώσεων χρήσης για τη διαχείριση προγραμμάτων παρουσιάζει τις λειτουργίες που είναι διαθέσιμες στον ρόλο Programmer, ο οποίος αποτελεί τον κύριο υπεύθυνο για τη δημιουργία και τη συνολική διαχείριση των κινηματογραφικών προγραμμάτων. Ο Programmer έχει τη δυνατότητα να δημιουργεί νέα προγράμματα μέσω της περίπτωσης χρήσης *Create Program*, ορίζοντας τα βασικά χαρακτηριστικά τους, όπως το όνομα, την περιγραφή και τις ημερομηνίες διεξαγωγής.

Επιπλέον, ο Programmer μπορεί να τροποποιεί υπάρχοντα προγράμματα μέσω της περίπτωσης χρήσης *Update Program*, καθώς και να τα διαγράφει όταν αυτά βρίσκονται σε επιτρεπτή κατάσταση, μέσω της περίπτωσης χρήσης *Delete Program*. Το διάγραμμα περιλαμβάνει επίσης τις λειτουργίες *Add Programmer* και *Add Staff*, οι οποίες επιτρέπουν την ανάθεση ρόλων σε άλλους αυθεντικοποιημένους χρήστες, υποστηρίζοντας τη συνεργατική διαχείριση ενός προγράμματος.

Τέλος, μέσω της περίπτωσης χρήσης *Change Program Status*, ο Programmer ελέγχει τη μετάβαση του προγράμματος μεταξύ των διαφορετικών καταστάσεων του κύκλου ζωής του (π.χ. CREATED, SUBMISSION, REVIEW, ANNOUNCED), σύμφωνα με τους περιορισμούς που θέτει το σύστημα. Το διάγραμμα αποτυπώνει τον κεντρικό ρόλο του Programmer και τον πλήρη έλεγχο που διαθέτει στη διαχείριση των προγραμμάτων, διασφαλίζοντας την ορθή και ελεγχόμενη λειτουργία του συστήματος.

3ο διάγραμμα Screening Management



Σχολιασμός Διαγράμματος Περιπτώσεων Χρήσης – Διαχείριση Προβολών (Use Case Diagram 3)

Το διάγραμμα περιπτώσεων χρήσης για τη διαχείριση προβολών απεικονίζει τις λειτουργίες που σχετίζονται με τον πλήρη κύκλο ζωής μιας προβολής (screening) και τον τρόπο με τον οποίο αυτές κατανέμονται μεταξύ των βασικών ρόλων του συστήματος. Στο διάγραμμα

συμμετέχουν οι ρόλοι Submitter, Staff και Programmer, καθένας εκ των οποίων διαθέτει διακριτές αρμοδιότητες και δικαιώματα πρόσβασης.

Ο ρόλος του Submitter είναι υπεύθυνος για τη δημιουργία και διαχείριση των προβολών που υποβάλλει. Μέσω των περιπτώσεων χρήσης *Create Screening* και *Update Screening*, ο Submitter μπορεί να καταχωρεί και να τροποποιεί στοιχεία μιας προβολής όσο αυτή βρίσκεται σε επιτρεπτή κατάσταση. Επιπλέον, έχει τη δυνατότητα να υποβάλει (*Submit Screening*), να αποσύρει (*Withdraw Screening*) ή να πραγματοποιήσει την τελική υποβολή (*Final Submit Screening*) μιας προβολής, ανάλογα με το στάδιο στο οποίο βρίσκεται το αντίστοιχο πρόγραμμα.

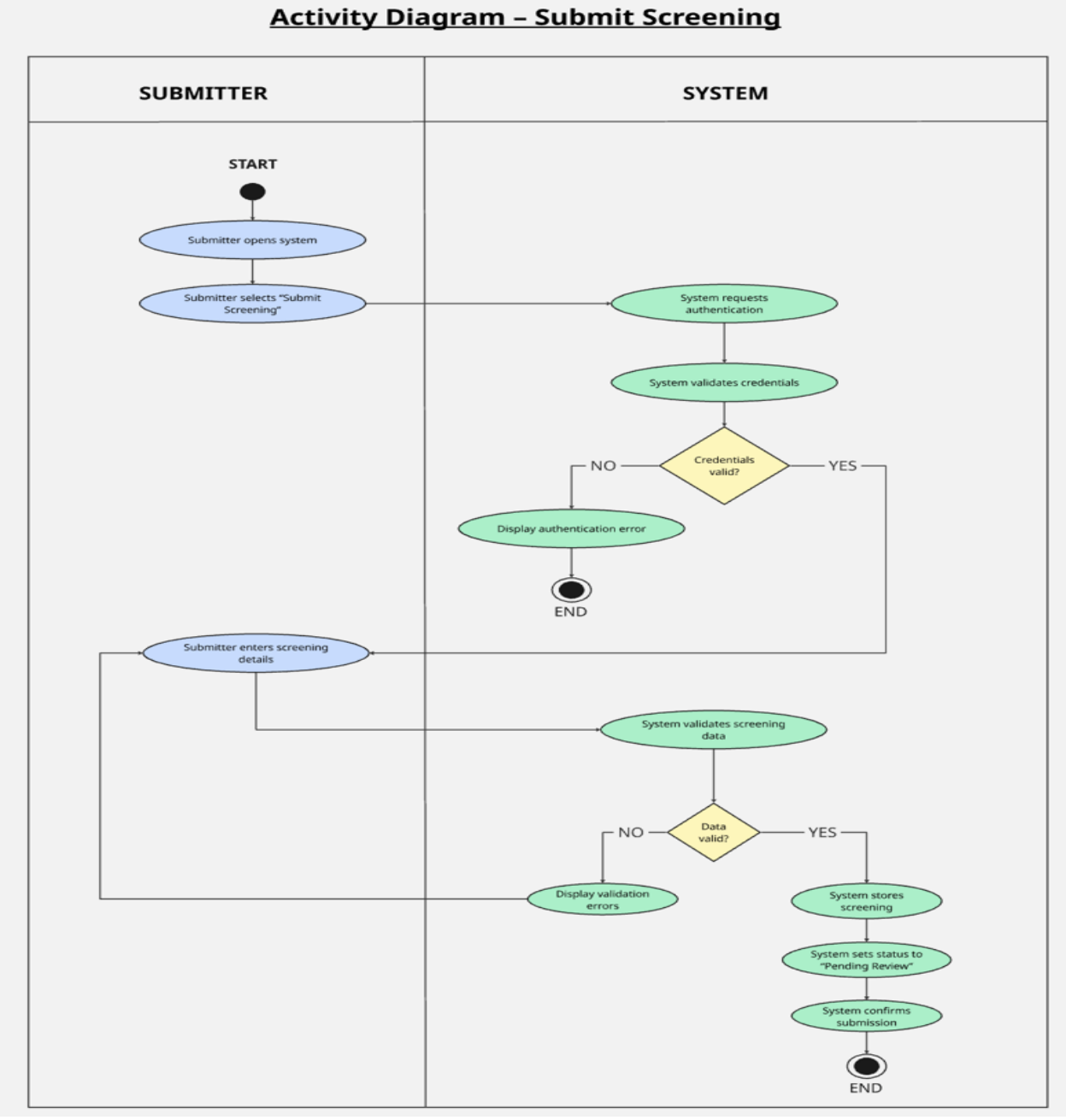
Ο Staff συμμετέχει στη διαδικασία αξιολόγησης των προβολών και έχει πρόσβαση μόνο στις προβολές που του έχουν ανατεθεί. Μέσω των περιπτώσεων χρήσης *View Assigned Screenings* και *Review Screenings*, ο Staff μπορεί να προβάλλει τις ανατεθειμένες προβολές και να καταχωρεί αξιολογήσεις και σχόλια, υποστηρίζοντας τη διαδικασία λήψης αποφάσεων.

Τέλος, ο Programmer έχει τον ρόλο του διαχειριστή της διαδικασίας και είναι υπεύθυνος για την ανάθεση προσωπικού μέσω της περίπτωσης χρήσης *Assign Staff*, καθώς και για τις τελικές αποφάσεις σχετικά με τις προβολές. Μέσω των περιπτώσεων χρήσης *Approve Screening*, *Reject Screening* και *Accept Screening*, ο Programmer καθορίζει αν μια προβολή θα προχωρήσει στο επόμενο στάδιο, θα απορριφθεί ή θα ενταχθεί οριστικά στο πρόγραμμα.

Το διάγραμμα αποτυπώνει με σαφήνεια τον διαχωρισμό ρόλων και ευθυνών, καθώς και τη συνεργασία μεταξύ των χρηστών για την ομαλή και ελεγχόμενη διαχείριση των προβολών, σύμφωνα με τους κανόνες και τις καταστάσεις που ορίζει το σύστημα.

3.4 Διαγράμματα Δραστηριοτήτων (Activity Diagrams)

1ο διάγραμμα submit screening



Σχολιασμός 1ου Διαγράμματος Δραστηριοτήτων – Υποβολή Προβολής (Activity Diagram – Submit Screening)

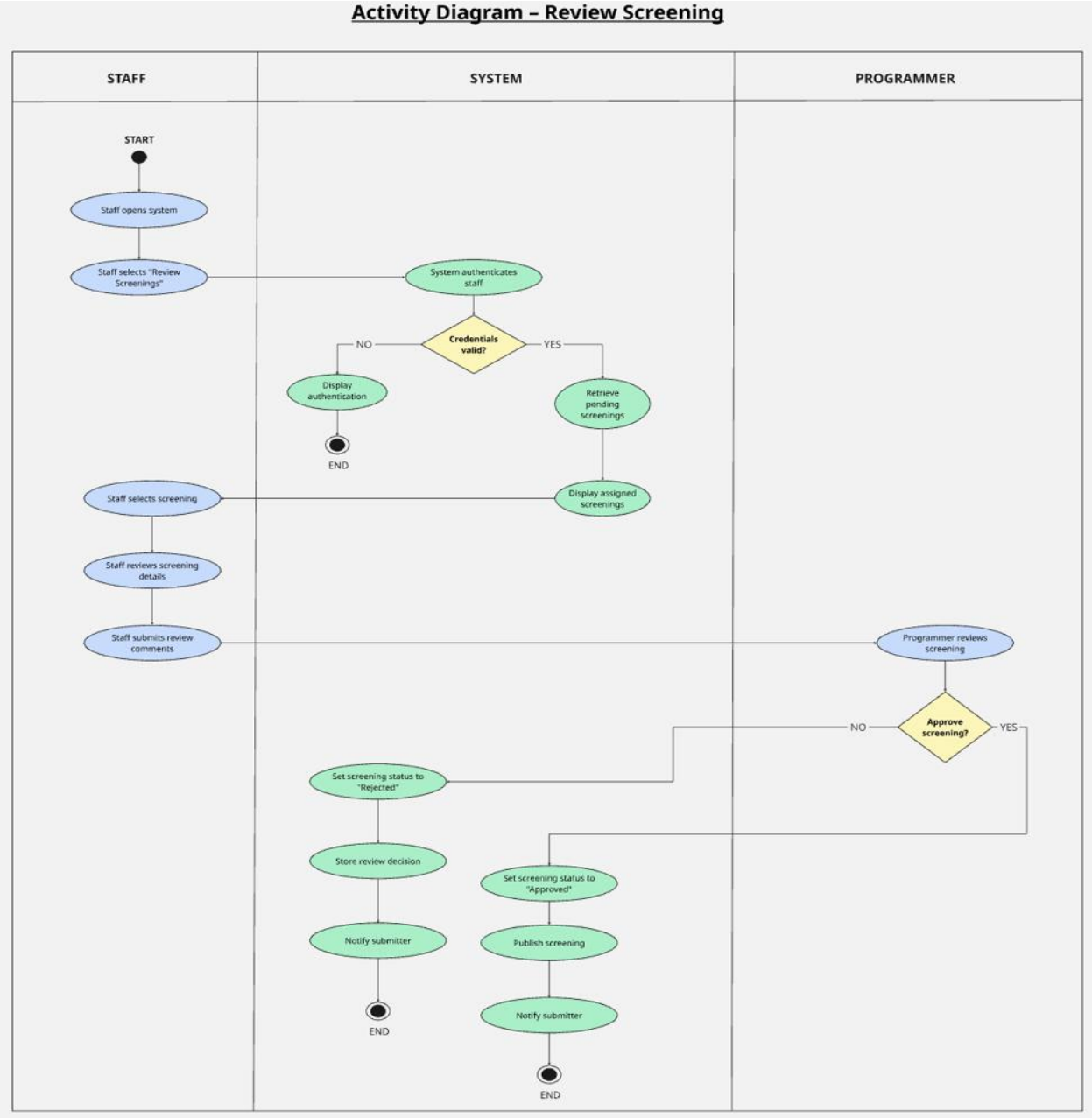
Το διάγραμμα δραστηριοτήτων απεικονίζει τη ροή ενεργειών που ακολουθείται κατά τη διαδικασία υποβολής μιας προβολής από τον ρόλο Submitter, καθώς και την αλληλεπίδρασή του με το σύστημα. Το διάγραμμα είναι οργανωμένο σε δύο διακριτές λωρίδες (swimlanes), οι οποίες διαχωρίζουν με σαφήνεια τις ενέργειες του χρήστη από τις ενέργειες του συστήματος.

Η διαδικασία ξεκινά όταν ο Submitter εισέρχεται στο σύστημα και επιλέγει την ενέργεια «Submit Screening». Στη συνέχεια, το σύστημα ζητά την αυθεντικοποίηση του χρήστη και πραγματοποιεί έλεγχο των διαπιστευτηρίων του. Σε περίπτωση αποτυχίας της αυθεντικοποίησης, η διαδικασία τερματίζεται με την εμφάνιση σχετικού μηνύματος σφάλματος. Αν τα στοιχεία είναι έγκυρα, ο Submitter προχωρά στην εισαγωγή των στοιχείων της προβολής.

Ακολούθως, το σύστημα ελέγχει την εγκυρότητα των δεδομένων που έχουν εισαχθεί. Αν εντοπιστούν σφάλματα ή ελλιπή στοιχεία, εμφανίζονται μηνύματα επικύρωσης και ο χρήστης καλείται να διορθώσει τα δεδομένα. Σε περίπτωση επιτυχούς επικύρωσης, το σύστημα αποθηκεύει την προβολή, ενημερώνει την κατάστασή της σε «Pending Review» και επιβεβαιώνει την ολοκλήρωση της υποβολής.

Το διάγραμμα αποτυπώνει με σαφήνεια τα σημεία ελέγχου, τις εναλλακτικές ροές (επιτυχία/αποτυχία) και τη συνεργασία μεταξύ χρήστη και συστήματος, διασφαλίζοντας την ορθότητα και την αξιοπιστία της διαδικασίας υποβολής προβολών.

2ο διάγραμμα Review Screening



Σχολιασμός 2ου Διαγράμματος Δραστηριοτήτων – Αξιολόγηση Προβολής (Activity Diagram – Review Screening)

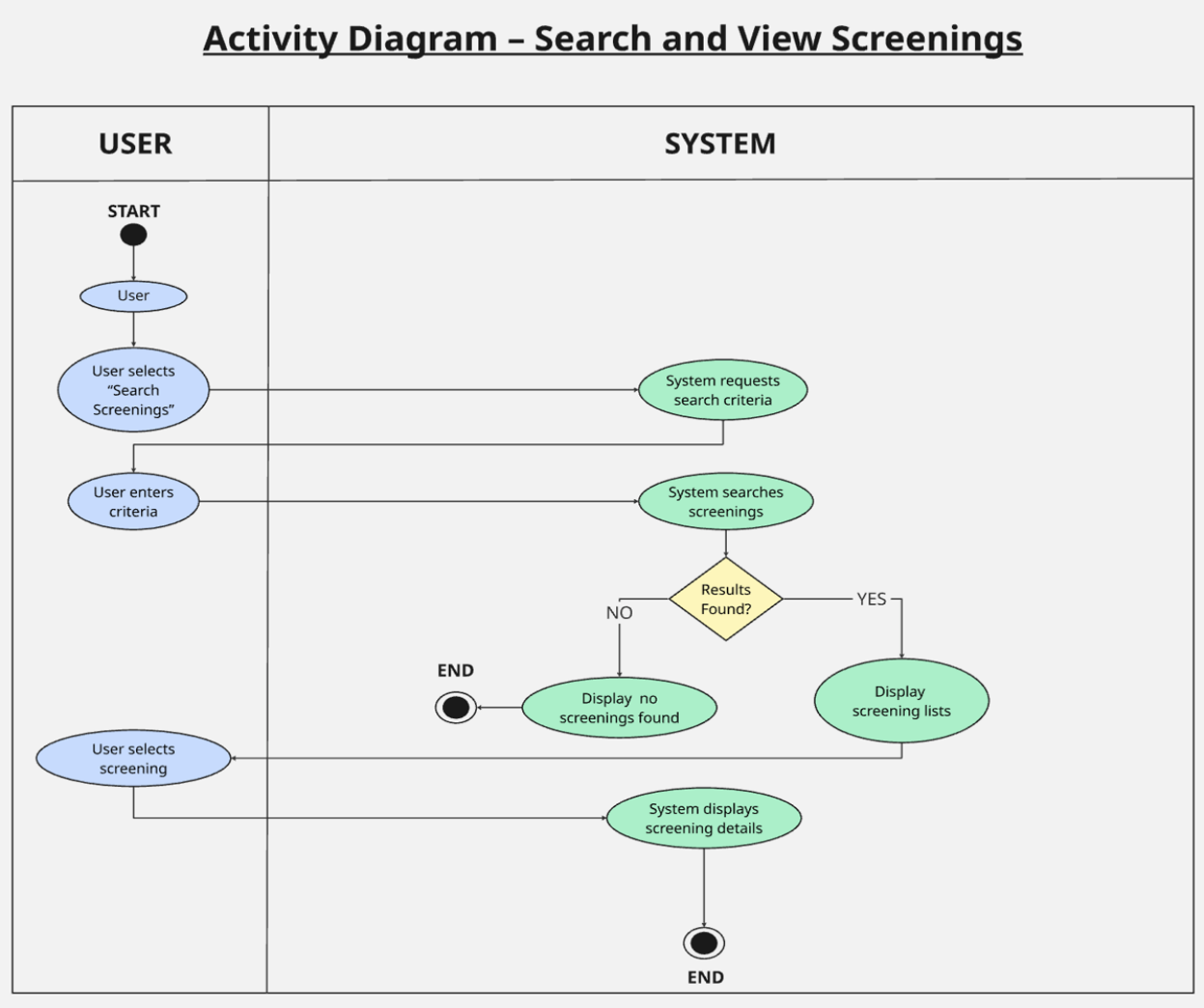
Το διάγραμμα δραστηριοτήτων απεικονίζει τη διαδικασία αξιολόγησης μιας προβολής και τη συνεργασία μεταξύ των ρόλων Staff, System και Programmer. Η ροή οργανώνεται σε τρεις διακριτές λωρίδες (swimlanes), ώστε να γίνεται σαφής ο διαχωρισμός των ενεργειών κάθε ρόλου και των αυτοματοποιημένων λειτουργιών του συστήματος.

Η διαδικασία ξεκινά με την είσοδο του Staff στο σύστημα και την επιλογή της λειτουργίας «Review Screenings». Το σύστημα προχωρά σε αυθεντικοποίηση του χρήστη και, σε περίπτωση αποτυχίας, η διαδικασία τερματίζεται με εμφάνιση κατάλληλου μηνύματος. Εφόσον η αυθεντικοποίηση είναι επιτυχής, το σύστημα ανακτά και εμφανίζει τις προβολές που έχουν ανατεθεί στο συγκεκριμένο μέλος του Staff.

Στη συνέχεια, ο Staff επιλέγει μία προβολή, μελετά τα στοιχεία της και υποβάλλει τα σχόλια και την αξιολόγησή του. Τα δεδομένα αυτά αποθηκεύονται από το σύστημα και προωθούνται στον Programmer, ο οποίος είναι υπεύθυνος για τη λήψη της τελικής απόφασης. Ο Programmer εξετάζει την προβολή και αποφασίζει αν θα την εγκρίνει ή θα την απορρίψει.

Σε περίπτωση απόρριψης, το σύστημα ενημερώνει την κατάσταση της προβολής σε «Rejected», αποθηκεύει την απόφαση και ειδοποιεί τον Submitter. Αντίθετα, σε περίπτωση έγκρισης, η προβολή χαρακτηρίζεται ως «Approved», δημοσιεύεται στο σύστημα και ο Submitter ενημερώνεται για το αποτέλεσμα. Το διάγραμμα αποτυπώνει με σαφήνεια τα εναλλακτικά μονοπάτια της διαδικασίας και αναδεικνύει τον ελεγχόμενο και ιεραρχικό χαρακτήρα της αξιολόγησης προβολών.

3ο διάγραμμα Search and View Screenings



Σχολιασμός Διαγράμματος Δραστηριοτήτων – Αναζήτηση και Προβολή Προβολών (Activity Diagram – Search and View Screenings)

Το διάγραμμα δραστηριοτήτων παρουσιάζει τη ροή ενεργειών για την αναζήτηση και προβολή προβολών από τον ρόλο User, καθώς και τις αντίστοιχες λειτουργίες που εκτελεί το σύστημα. Η διαδικασία διαχωρίζεται σε δύο λωρίδες (swimlanes), οι οποίες απεικονίζουν τις ενέργειες του χρήστη και τις αυτοματοποιημένες λειτουργίες του συστήματος.

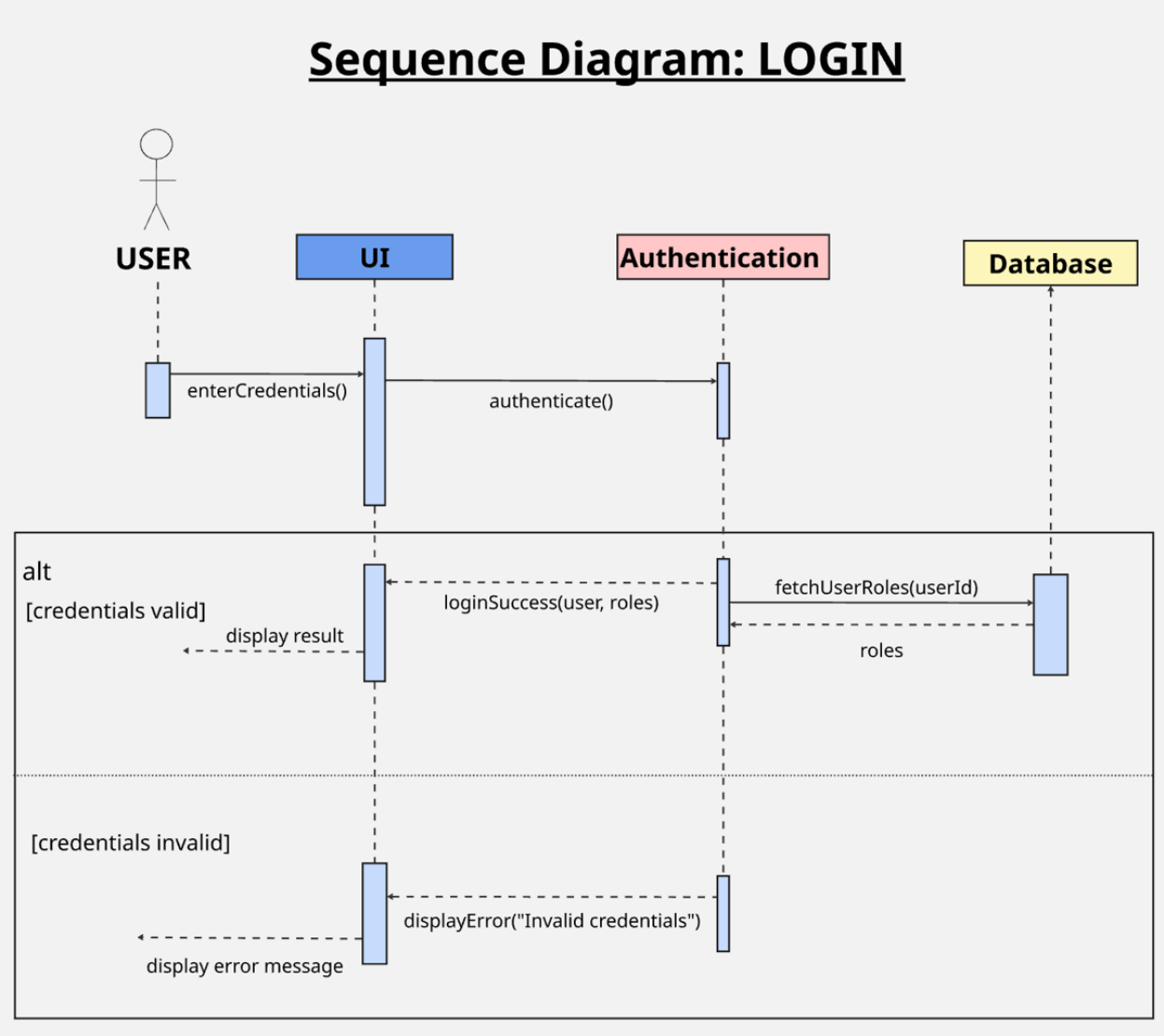
Η ροή ξεκινά όταν ο χρήστης επιλέγει τη λειτουργία «Search Screenings» και εισάγει τα επιθυμητά κριτήρια αναζήτησης, όπως τίτλο ταινίας, είδος ή χρονικό διάστημα. Το σύστημα επεξεργάζεται τα κριτήρια και πραγματοποιεί αναζήτηση στις διαθέσιμες προβολές. Στη συνέχεια, ελέγχεται αν υπάρχουν αποτελέσματα που ικανοποιούν τα κριτήρια αναζήτησης.

Σε περίπτωση που δεν εντοπιστούν προβολές, το σύστημα εμφανίζει σχετικό μήνυμα ενημέρωσης και η διαδικασία ολοκληρώνεται. Αντίθετα, εφόσον υπάρχουν διαθέσιμα αποτελέσματα, το σύστημα εμφανίζει τη λίστα των προβολών στον χρήστη. Ο χρήστης μπορεί στη συνέχεια να επιλέξει μία συγκεκριμένη προβολή, οπότε το σύστημα προβάλλει τα αναλυτικά στοιχεία της.

Το διάγραμμα αποτυπώνει με σαφήνεια τις εναλλακτικές ροές της διαδικασίας (εύρεση ή μη αποτελεσμάτων) και αναδεικνύει την απλότητα και την ευχρηστία της λειτουργίας αναζήτησης, η οποία αποτελεί βασικό στοιχείο της δημόσιας και αυθεντικοποιημένης πρόσβασης στο σύστημα.

3.5 Διαγράμματα Ακολουθίας (Sequence Diagrams)

1ο διάγραμμα LOGIN



Σχολιασμός Διαγράμματος Ακολουθίας – Σύνδεση Χρήστη (Sequence Diagram – Login)

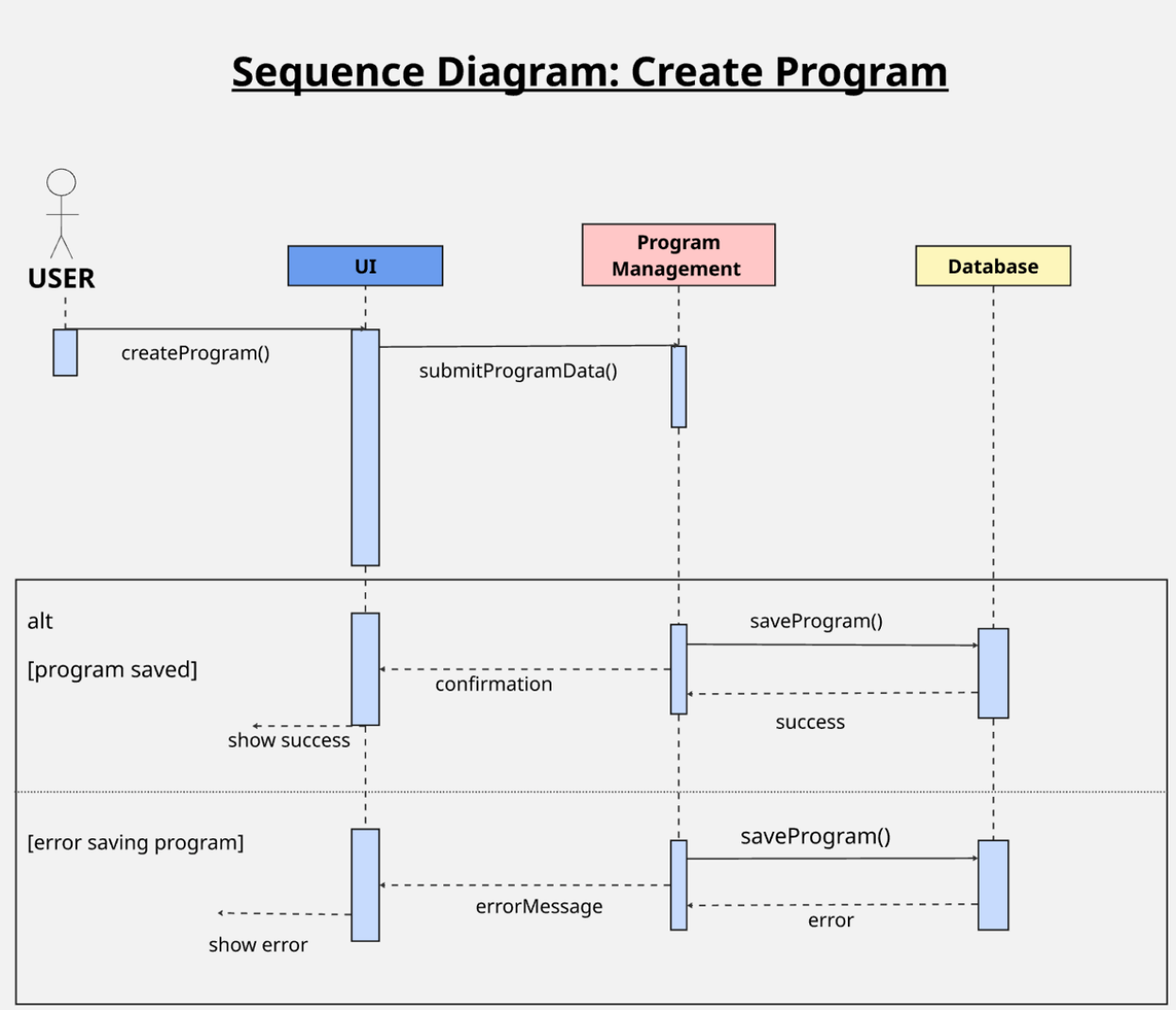
Το διάγραμμα ακολουθίας απεικονίζει τη διαδικασία σύνδεσης (login) ενός χρήστη στο σύστημα και την αλληλεπίδραση μεταξύ των βασικών συστατικών του συστήματος, συγκεκριμένα του User, του User Interface (UI), της υπηρεσίας Authentication και της Database. Το διάγραμμα παρουσιάζει τη χρονική σειρά των μηνυμάτων που ανταλλάσσονται κατά την αυθεντικοποίηση του χρήστη.

Η διαδικασία ξεκινά όταν ο χρήστης εισάγει τα διαπιστευτήριά του μέσω του UI. Το UI προωθεί τα στοιχεία αυτά στην υπηρεσία Authentication μέσω της κλήσης authenticate(). Η υπηρεσία Authentication επικοινωνεί με τη βάση δεδομένων για να επαληθεύσει την εγκυρότητα των διαπιστευτηρίων και, σε περίπτωση επιτυχούς ελέγχου, ανακτά τους ρόλους που αντιστοιχούν στον χρήστη μέσω της κλήσης fetchUserRoles(userId).

Στο εναλλακτικό σενάριο επιτυχούς αυθεντικοποίησης (credentials valid), η υπηρεσία Authentication επιστρέφει στο UI την επιτυχή σύνδεση μαζί με τους ρόλους του χρήστη, και το UI εμφανίζει το αντίστοιχο αποτέλεσμα στον χρήστη. Αντίθετα, στο σενάριο αποτυχίας αυθεντικοποίησης (credentials invalid), η υπηρεσία Authentication επιστρέφει μήνυμα σφάλματος και το UI εμφανίζει κατάλληλο μήνυμα αποτυχίας σύνδεσης.

Το διάγραμμα αναδεικνύει τον ρόλο της βάσης δεδομένων στην επαλήθευση των στοιχείων και στον καθορισμό των δικαιωμάτων πρόσβασης, καθώς και τον σαφή διαχωρισμό αρμοδιοτήτων μεταξύ διεπαφής χρήστη και επιχειρησιακής λογικής. Παράλληλα, αποτυπώνει με ακρίβεια τις εναλλακτικές ροές της διαδικασίας σύνδεσης και τον χειρισμό σφαλμάτων, συμβάλλοντας στην κατανόηση της λειτουργίας αυθεντικοποίησης του συστήματος.

2ο διάγραμμα Create Program



Σχολιασμός Διαγράμματος Ακολουθίας – Δημιουργία Προγράμματος (Sequence Diagram – Create Program)

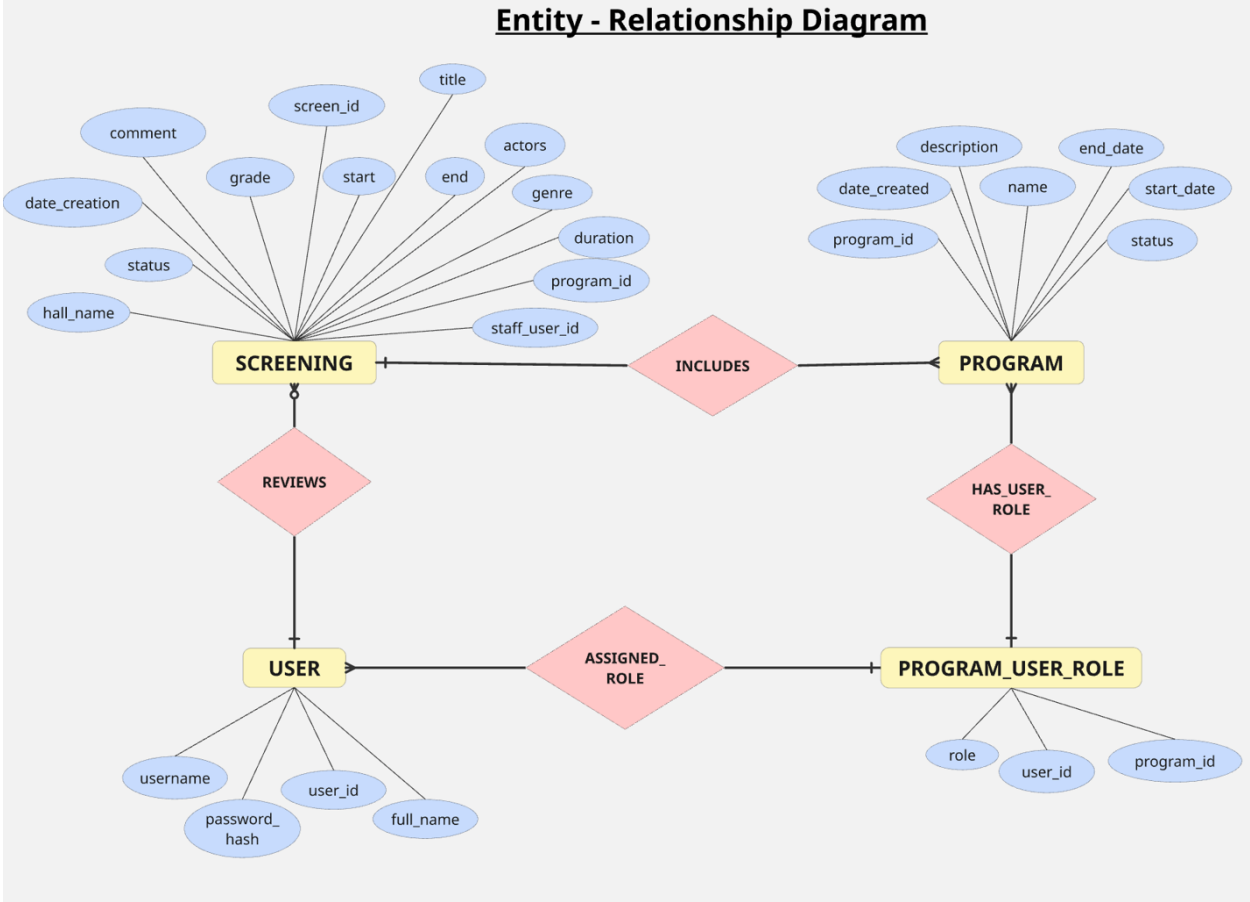
Το διάγραμμα ακολουθίας παρουσιάζει τη διαδικασία δημιουργίας ενός νέου κινηματογραφικού προγράμματος και την αλληλεπίδραση μεταξύ των βασικών στοιχείων του συστήματος, συγκεκριμένα του User, του User Interface (UI), της υπηρεσίας Program Management και της Database. Το διάγραμμα απεικονίζει τη χρονική ακολουθία των μηνυμάτων που ανταλλάσσονται κατά την εκτέλεση της συγκεκριμένης λειτουργίας.

Η διαδικασία ξεκινά όταν ο χρήστης αιτείται τη δημιουργία νέου προγράμματος μέσω του UI, ενεργοποιώντας τη λειτουργία createProgram(). Το UI συλλέγει τα απαραίτητα δεδομένα και τα προωθεί στο υποσύστημα Program Management μέσω της κλήσης submitProgramData(). Το υποσύστημα Program Management αναλαμβάνει την επιχειρησιακή λογική της λειτουργίας, ελέγχοντας την εγκυρότητα των δεδομένων και τη μοναδικότητα του ονόματος του προγράμματος.

Στο εναλλακτικό σενάριο επιτυχούς ολοκλήρωσης (program saved), το Program Management αποθηκεύει το πρόγραμμα στη βάση δεδομένων μέσω της κλήσης saveProgram() και λαμβάνει επιβεβαίωση επιτυχίας. Η επιβεβαίωση αυτή επιστρέφεται στο UI, το οποίο ενημερώνει τον χρήστη για την επιτυχή δημιουργία του προγράμματος. Αντίθετα, στο σενάριο αποτυχίας (error saving program), το σύστημα λαμβάνει μήνυμα σφάλματος από τη βάση δεδομένων και το προωθεί στο UI, το οποίο εμφανίζει κατάλληλο μήνυμα αποτυχίας στον χρήστη.

Το διάγραμμα αναδεικνύει τον διαχωρισμό αρμοδιοτήτων μεταξύ διεπαφής χρήστη, επιχειρησιακής λογικής και επιπέδου αποθήκευσης δεδομένων, καθώς και τον χειρισμό εναλλακτικών ροών και σφαλμάτων κατά τη δημιουργία ενός προγράμματος. Η απεικόνιση αυτή συμβάλλει στην κατανόηση της λειτουργίας και της αξιοπιστίας της διαδικασίας διαχείρισης προγραμμάτων στο σύστημα.

3.6 Διάγραμμα Οντοτήτων-Συσχετίσεων (Entity-Relationship (ER) diagram)



Σχολιασμός Διαγράμματος Οντοτήτων – Συσχετίσεων (Entity–Relationship Diagram)

Το διάγραμμα οντοτήτων–συσχετίσεων απεικονίζει τη δομή της βάσης δεδομένων του συστήματος διαχείρισης κινηματογράφου και τις σχέσεις μεταξύ των βασικών οντοτήτων. Ο σχεδιασμός στοχεύει στη σωστή οργάνωση των δεδομένων και στη διασφάλιση της ακεραιότητας και της συνέπειάς τους.

Η οντότητα PROGRAM αντιπροσωπεύει ένα κινηματογραφικό πρόγραμμα και περιλαμβάνει βασικά χαρακτηριστικά όπως το αναγνωριστικό προγράμματος, το όνομα, την περιγραφή, τις ημερομηνίες έναρξης και λήξης, την κατάσταση και την ημερομηνία δημιουργίας. Κάθε πρόγραμμα μπορεί να περιλαμβάνει πολλές προβολές, γεγονός που αποτυπώνεται μέσω της συσχέτισης INCLUDES με την οντότητα SCREENING.

Η οντότητα SCREENING αναπαριστά μια κινηματογραφική προβολή και περιλαμβάνει πληροφορίες όπως τίτλο ταινίας, είδος, διάρκεια, ηθοποιούς, ώρα έναρξης και λήξης, κατάσταση, αίθουσα προβολής, καθώς και στοιχεία αξιολόγησης (βαθμολογία και σχόλια). Κάθε προβολή ανήκει υποχρεωτικά σε ένα πρόγραμμα, ενώ συνδέεται με την οντότητα USER μέσω της συσχέτισης REVIEWS, η οποία επιτρέπει την καταχώρηση αξιολογήσεων από το προσωπικό.

Η οντότητα USER περιλαμβάνει τα στοιχεία των χρηστών του συστήματος, όπως μοναδικό αναγνωριστικό, όνομα χρήστη, κρυπτογραφημένο κωδικό πρόσβασης και πλήρες όνομα. Οι χρήστες μπορούν να έχουν διαφορετικούς ρόλους σε διαφορετικά προγράμματα, κάτι που υλοποιείται μέσω της ενδιάμεσης οντότητας PROGRAM_USER_ROLE.

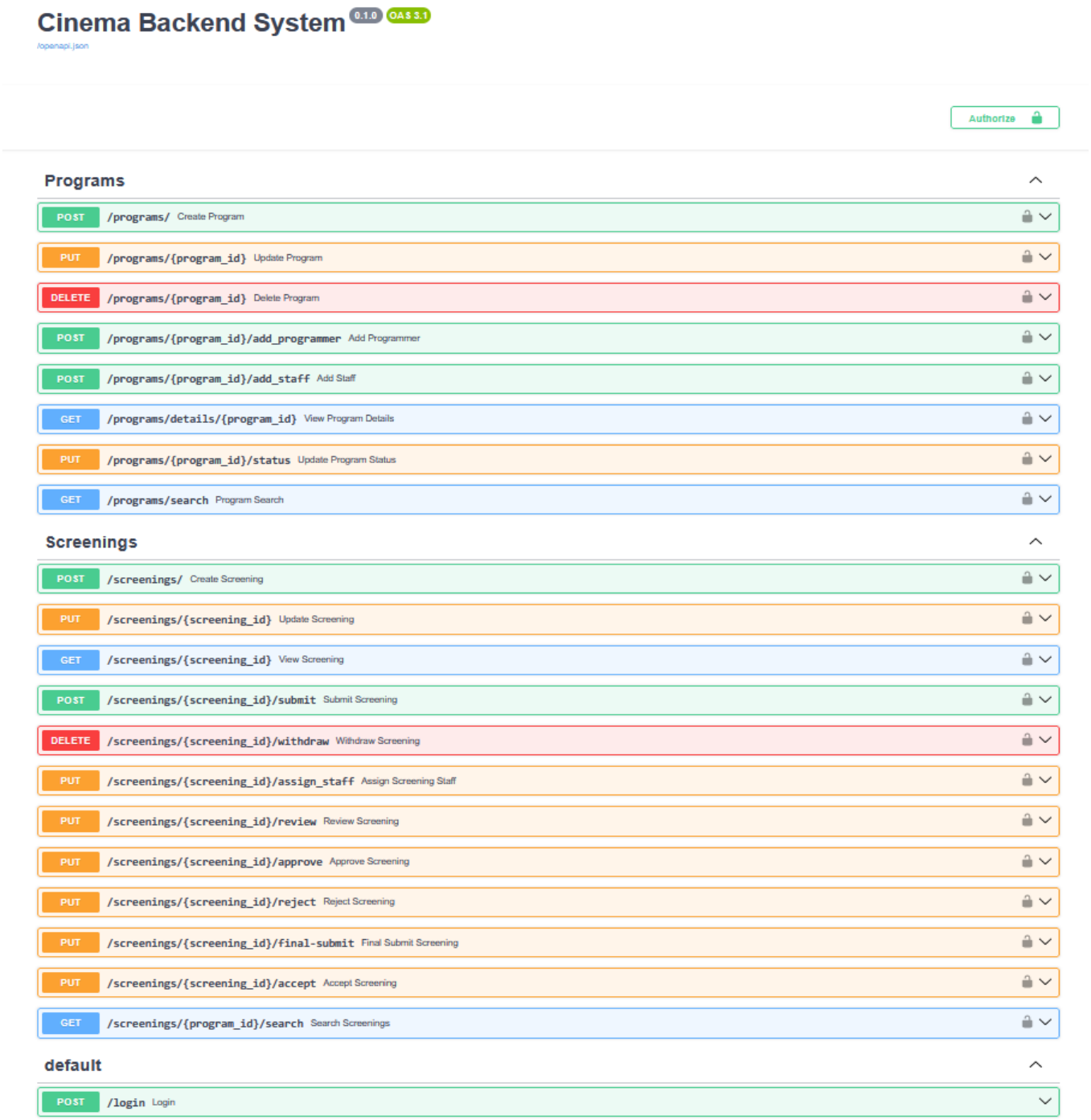
Η οντότητα PROGRAM_USER_ROLE λειτουργεί ως πίνακας συσχέτισης μεταξύ των οντοτήτων PROGRAM και USER και αποθηκεύει τον ρόλο που κατέχει ένας χρήστης σε ένα συγκεκριμένο πρόγραμμα (π.χ. Programmer, Staff, Submitter). Η ύπαρξή της επιτρέπει την υποστήριξη πολλαπλών ρόλων χρηστών σε διαφορετικά προγράμματα, διατηρώντας παράλληλα τον κανόνα ενός ρόλου ανά χρήστη και πρόγραμμα.

Συνολικά, το ER διάγραμμα αποτυπώνει έναν κανονικοποιημένο και επεκτάσιμο σχεδιασμό βάσης δεδομένων, ο οποίος υποστηρίζει πλήρως τις λειτουργικές απαιτήσεις του συστήματος και εξασφαλίζει αξιόπιστη αποθήκευση και ανάκτηση δεδομένων.

4. Υλοποίηση Συστήματος / Εφαρμογής

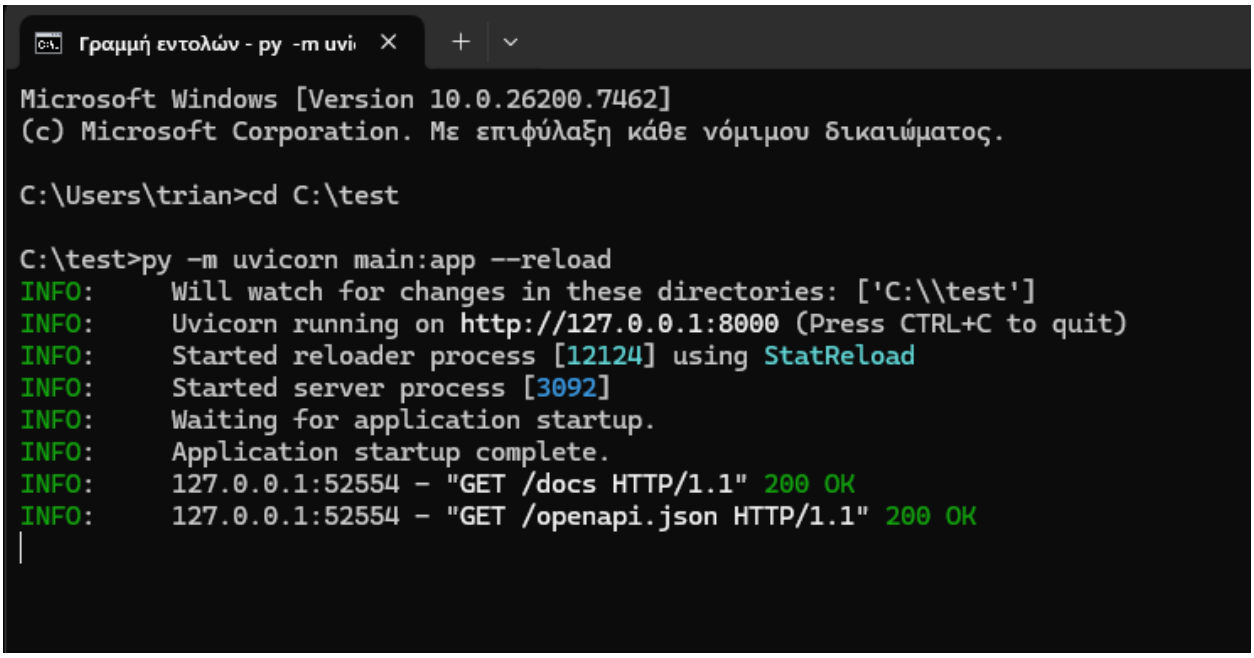
4.1 Περιβάλλον Ανάπτυξης και Έλεγχος REST API

Η ανάπτυξη της εφαρμογής πραγματοποιήθηκε στο περιβάλλον Visual Studio Code (VS Code), το οποίο χρησιμοποιήθηκε για τη συγγραφή και οργάνωση του πηγαίου κώδικα. Η υλοποίηση έγινε στη γλώσσα προγραμματισμού Python, με χρήση του πλαισίου FastAPI για την ανάπτυξη RESTful web services. Η βάση δεδομένων του συστήματος υλοποιήθηκε μέσω του XAMPP, το οποίο παρείχε το απαραίτητο περιβάλλον για τη διαχείριση του σχεσιακού συστήματος βάσεων δεδομένων.



Σχήμα 4.1: *Ενσωματωμένο περιβάλλον Swagger UI του FastAPI για τον έλεγχο των RESTful υπηρεσιών.*

Ο έλεγχος της λειτουργικότητας των RESTful υπηρεσιών πραγματοποιήθηκε μέσω του ενσωματωμένου web περιβάλλοντος του FastAPI (Swagger UI), το οποίο είναι διαθέσιμο στη διεύθυνση /docs. Μέσω του Swagger UI δοκιμάστηκαν τα βασικά API endpoints του συστήματος, όπως η δημιουργία και διαχείριση προγραμμάτων και προβολών, η αυθεντικοποίηση χρηστών και η αναζήτηση δεδομένων. Παράλληλα, χρησιμοποιήθηκε το τερματικό για την εκκίνηση του server και την παρακολούθηση της ορθής λειτουργίας της εφαρμογής.



Σχήμα 4.2: *Εκκίνηση και εκτέλεση του backend server FastAPI μέσω τερματικού.*

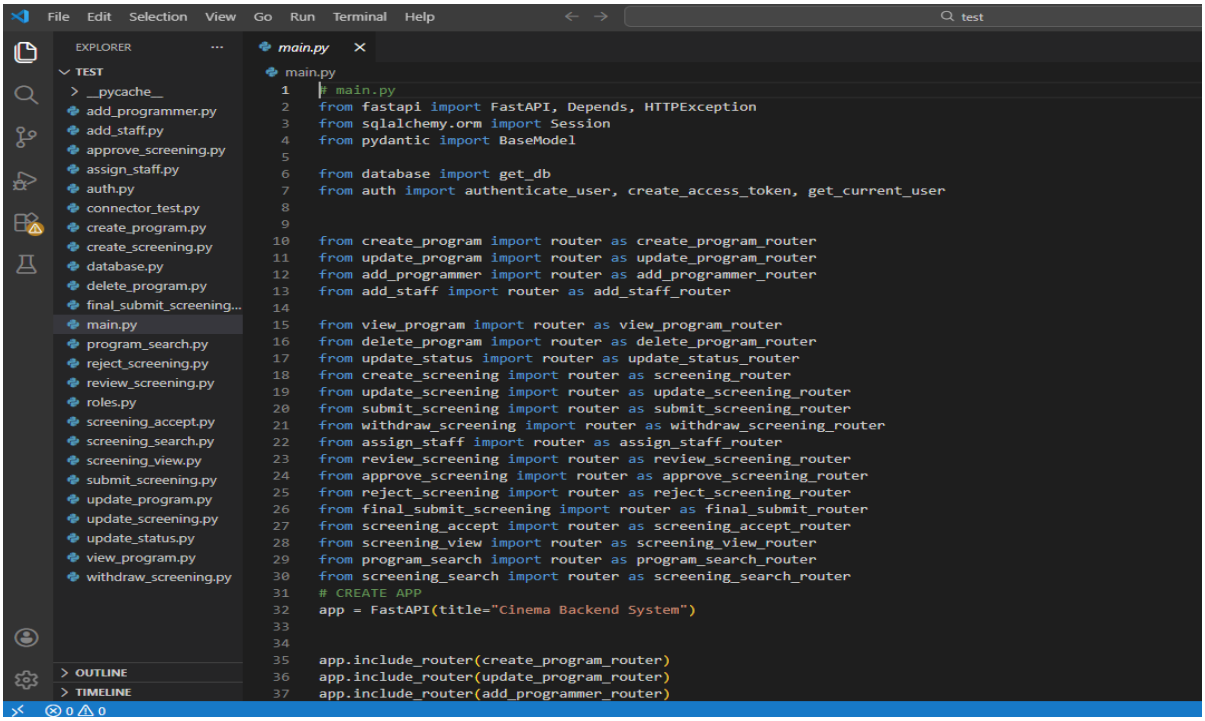
4.2 Υλοποίηση και Σχολιασμός

Η υλοποίηση του συστήματος βασίστηκε σε αρθρωτή αρχιτεκτονική, με σαφή διαχωρισμό των λειτουργιών της εφαρμογής. Το backend αναπτύχθηκε με χρήση FastAPI και περιλαμβάνει RESTful endpoints που υλοποιούν τη βασική επιχειρησιακή λογική του συστήματος, όπως τη διαχείριση χρηστών, προγραμμάτων και προβολών, καθώς και τον έλεγχο πρόσβασης βάσει ρόλων.

Η εφαρμογή διασυνδέεται με τη βάση δεδομένων που υλοποιήθηκε μέσω XAMPP, εξασφαλίζοντας την αποθήκευση και ανάκτηση των δεδομένων με συνέπεια και αξιοπιστία. Κατά την υλοποίηση δόθηκε έμφαση:

- στην επικύρωση των δεδομένων εισόδου,
- στη διαχείριση των καταστάσεων (states) προγραμμάτων και προβολών,
- στον χειρισμό σφαλμάτων και την επιστροφή κατάλληλων HTTP κωδικών κατάστασης,
- στη βασική ασφάλεια μέσω αυθεντικοποίησης χρηστών.

Η ορθότητα της υλοποίησης επαληθεύτηκε μέσω δοκιμών των λειτουργιών του συστήματος στο περιβάλλον ανάπτυξης, διασφαλίζοντας ότι η εφαρμογή λειτουργεί σύμφωνα με τις απαιτήσεις που ορίστηκαν κατά τη φάση της ανάλυσης και του σχεδιασμού.



Σχήμα 4.3: Δομή υλοποίησης του backend συστήματος σε Python. Κάθε αρχείο αντιστοιχεί σε συγκεκριμένη λειτουργία ή REST endpoint της εφαρμογής.

5. Συμπεράσματα

5.1 Εμπειρία που αποκτήθηκε από το έργο

Η υλοποίηση του συστήματος διαχείρισης κινηματογράφου αποτέλεσε μια ολοκληρωμένη εμπειρία εφαρμογής των βασικών αρχών της μηχανικής λογισμικού. Μέσα από το έργο έγινε κατανοητή η σημασία της ανάλυσης απαιτήσεων, του σωστού σχεδιασμού μέσω διαγραμμάτων UML και της σταδιακής μετάβασης από τη θεωρία στην πρακτική υλοποίηση. Ιδιαίτερη αξία είχε η ανάπτυξη RESTful υπηρεσιών με χρήση του FastAPI, καθώς και η σύνδεσή τους με βάση δεδομένων, προσφέροντας εμπειρία σε σύγχρονες τεχνολογίες backend ανάπτυξης.

Παράλληλα, το έργο συνέβαλε στην εξοικείωση με την έννοια των ρόλων χρηστών, της διαχείρισης καταστάσεων (states) και της ασφάλειας σε εφαρμογές web, ενισχύοντας την κατανόηση της λειτουργίας πραγματικών πληροφοριακών συστημάτων.

5.2 Προβλήματα και δυσκολίες που αντιμετωπίστηκαν

Κατά τη διάρκεια της ανάπτυξης του συστήματος παρουσιάστηκαν ορισμένες δυσκολίες, κυρίως στον σχεδιασμό της βάσης δεδομένων και στη σωστή υλοποίηση των μεταβάσεων μεταξύ των καταστάσεων των προγραμμάτων και των προβολών. Επίσης, η διαχείριση των διαφορετικών ρόλων χρηστών και των αντίστοιχων δικαιωμάτων πρόσβασης απαιτούσε προσεκτικό σχεδιασμό, ώστε να τηρούνται οι περιορισμοί της εκφώνησης.

Προκλήσεις παρουσιάστηκαν και στον έλεγχο της εφαρμογής, κυρίως στον εντοπισμό και τη διόρθωση σφαλμάτων κατά την επικοινωνία μεταξύ των REST endpoints και της βάσης δεδομένων. Ωστόσο, μέσω της χρήσης του ενσωματωμένου περιβάλλοντος δοκιμών του FastAPI και της σταδιακής ανάπτυξης του κώδικα, τα προβλήματα αυτά αντιμετωπίστηκαν επιτυχώς.

5.3 Βέλτιστες πρακτικές και συμπεράσματα για μελλοντικά έργα

Κατά την υλοποίηση του έργου αναδείχθηκε η σημασία της αρθρωτής σχεδίασης και του διαχωρισμού της λειτουργικότητας σε επιμέρους ενότητες, κάτι που διευκόλυνε την ανάπτυξη και τη συντήρηση του κώδικα. Η χρήση σαφών ονομάτων αρχείων και λειτουργιών, καθώς και η τήρηση σταθερής δομής, αποδείχθηκαν ιδιαίτερα ωφέλιμες.

Επιπλέον, η συστηματική χρήση διαγραμμάτων πριν την υλοποίηση βοήθησε στον εντοπισμό πιθανών σχεδιαστικών προβλημάτων σε πρώιμο στάδιο. Για μελλοντικά έργα, η χρήση συστημάτων ελέγχου εκδόσεων και αυτοματοποιημένων δοκιμών θα μπορούσε να βελτιώσει περαιτέρω την ποιότητα και την οργάνωση της ανάπτυξης. Συνολικά, το έργο παρείχε πολύτιμη εμπειρία και γνώσεις που μπορούν να εφαρμοστούν σε μελλοντικά έργα ανάπτυξης λογισμικού.

6.Πηγές

1.FastAPI επίσημη τεκμηρίωση του FastAPI για την ανάπτυξη RESTful web services σε Pyhton

2.Python. επίσημη τεκμηρίωση της γλώσσας προγραμματισμού Python

3.Swagger / OpenAPI – Τεκμηρίωση για το Swagger UI και τη χρήση του για τον έλεγχο και την τεκμηρίωση REST API.

4.XAMPP – Τεκμηρίωση του XAMPP για τη διαχείριση τοπικού server και βάσης δεδομένων.

5.MySQL – Τεκμηρίωση του συστήματος διαχείρισης σχεσιακών βάσεων δεδομένων MySQL.

6.Sommerville, I. *Software Engineering*, Pearson Education – Βασικές αρχές ανάλυσης, σχεδίασης και ανάπτυξης λογισμικού.

7.UML Documentation – Unified Modeling Language για τη μοντελοποίηση συστημάτων (Use Case, Activity, Sequence, ER diagrams).

8.RESTful Web Services, Leonard Richardson, Sam Ruby, O'Reilly Media. Αρχές και βέλτιστες πρακτικές για την ανάπτυξη RESTful υπηρεσιών.

9.Applying UML and Patterns, Craig Larman, Prentice Hall. Αναλυτική παρουσίαση της UML και της χρήσης διαγραμμάτων Use Case, Activity και Sequence στον αντικειμενοστραφή σχεδιασμό

7.GitHub link

https://github.com/christriant4/cinema-management-system